SE Lab Week 7

Fuzzing for Input Validation Bugs

NAME: KUSHITA

SRN: PES1UG24CS247

SECTION: E

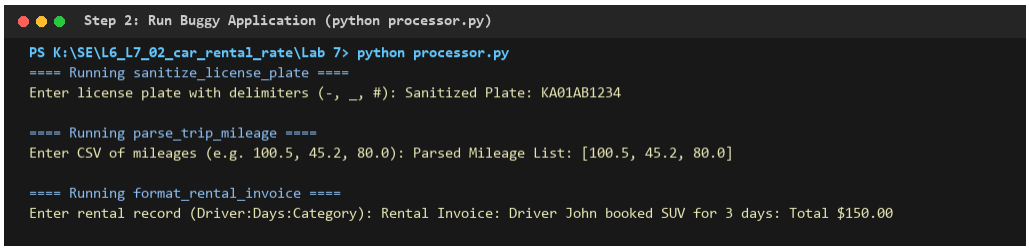
1. Introduction

The objective of this lab is to explore fuzz testing as a dynamic security and quality assurance technique by finding and fixing input validation bugs in a car rental rate processor application using the Hypothesis fuzzing library in Python. While standard unit tests only check specific inputs that a developer anticipates, fuzz testing generates hundreds of unexpected, random, malformed, or boundary inputs to discover edge cases that cause crashes or uncaught exceptions. In this lab, we test three routines: license plate sanitization, trip mileage parsing from CSV format, and rental invoice string formatting. We use Hypothesis to detect uncaught exceptions, analyze why each failure happened, and then harden the code using defensive programming techniques so that the functions handle all invalid inputs gracefully.

2. Buggy Application Execution

The initial application (processor.py) was provided with basic implementations for the three routines. When run interactively in the terminal with expected, well-formed sample data, the program executes without visible errors. However, it makes strong assumptions about input types and formats without any input validation.

Command: python processor.py



3. Property-Based Fuzz Testing with Hypothesis

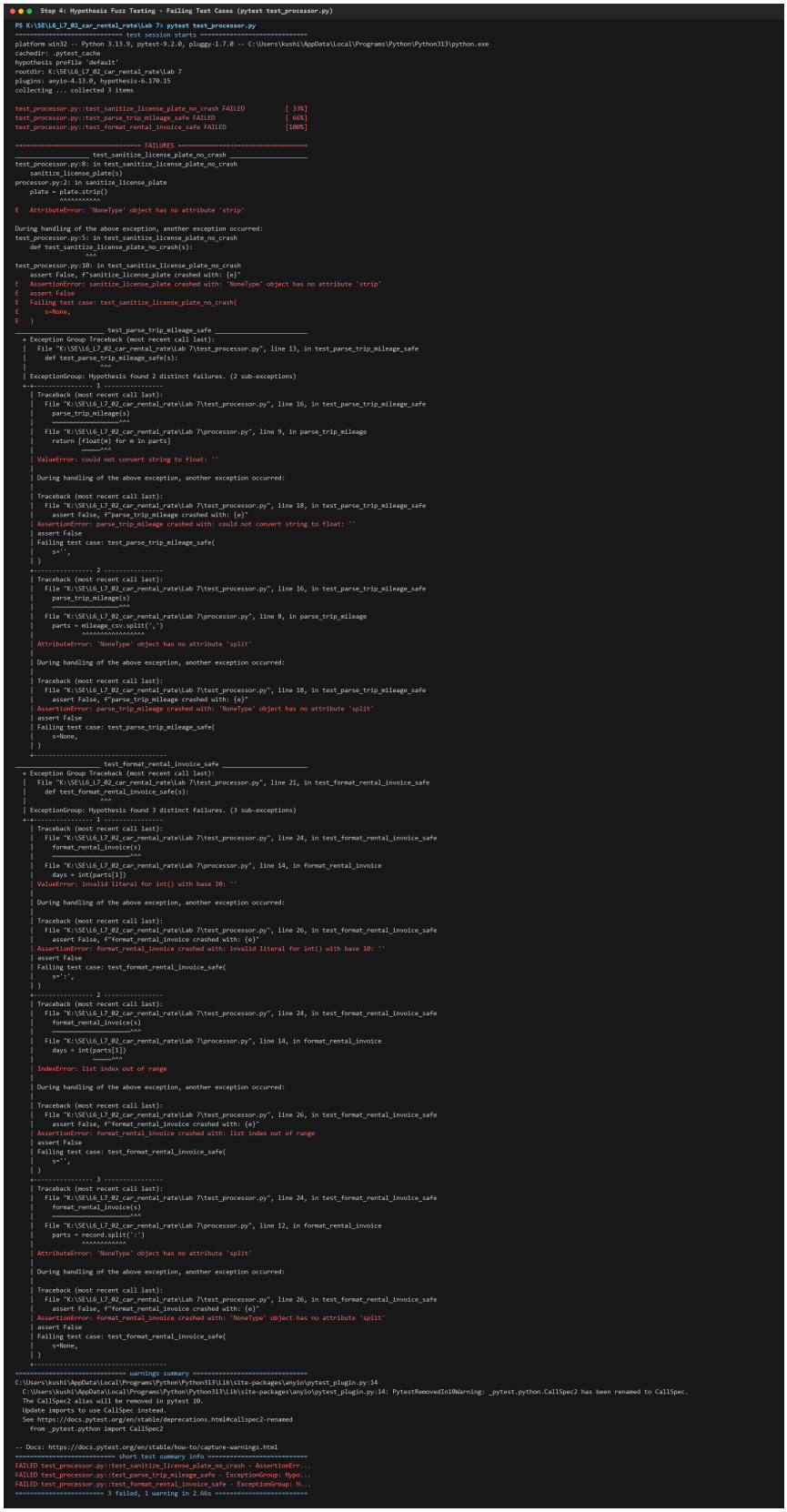
To systematically test the resilience of the routines, property-based tests were implemented in test_processor.py using the Hypothesis library. Using the @given(st.text() | st.none()) strategy, Hypothesis fed arbitrary Unicode text strings as well as None into each target function to verify that none of the functions crash or throw unhandled exceptions under any input.

Observed Failures during Initial Fuzz Run:

- **sanitize_license_plate:** Crashed with AttributeError: 'NoneType' object has no attribute 'strip' when generated input was None.
- **parse_trip_mileage:** Crashed with AttributeError on None and ValueError: could not convert string to float: '' when given empty strings or non-numeric elements.
- **format_rental_invoice:** Crashed with AttributeError on None, IndexError: list index out of range on inputs with fewer than two colons, and ValueError when days could not be converted to an integer.

Failing Test Cases Terminal Output:

Running pytest test_processor.py on the unhardened application exposed all three distinct failures with minimized failing examples generated by Hypothesis:



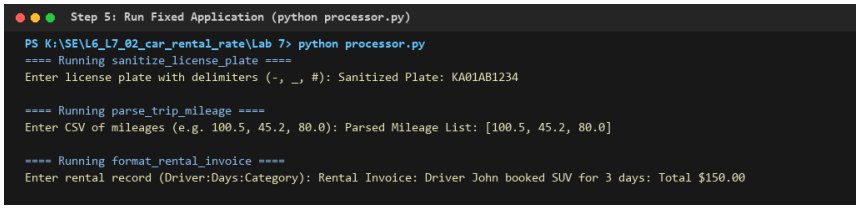
4. Defensive Hardening and Bug Fixes

To make the code robust against any arbitrary input, processor.py was modified using defensive programming:

- **Type Checking and Null Safety:** Each function checks if the input is None or not a string (isinstance(..., str)), returning safe default values (an empty string, empty list, or descriptive message).
- **Delimiter Length Verification:** format_rental_invoice verifies that splitting by colon yields at least 3 parts before attempting index lookups, avoiding IndexError.
- **Exception-Safe Type Parsing:** parse_trip_mileage parses floats inside a try-except ValueError block, discarding empty tokens or negative values. format_rental_invoice catches integer conversion errors and validates that days are non-negative.

Execution of Hardened Application:

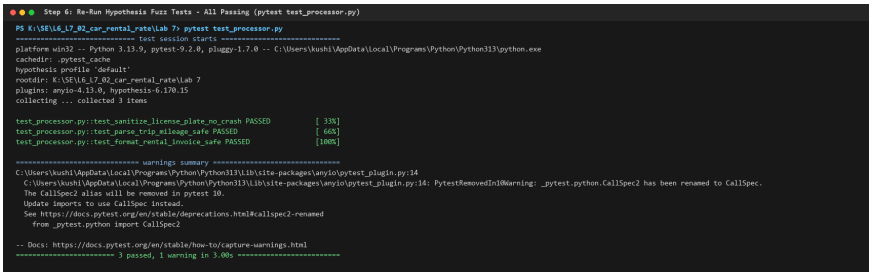
Command: python processor.py



5. Verification with Fuzz Tests

After hardening the application, pytest test_processor.py was re-executed. Hypothesis generated hundreds of random inputs across all three test functions, and all tests passed without triggering any exceptions.

Command: pytest test_processor.py



Testing and Hardening Summary:

Function	Identified Bug	Fix Applied	Fuzz Result
sanitize_license_plate	AttributeError on None	Added None/type guard returning ''	PASSED
parse_trip_mileage	ValueError on '', AttributeError on None	Filter blank tokens, try-except float conversion	PASSED
format_rental_invoice	IndexError, ValueError, AttributeError	Checked parts length, safe int parsing with validation	PASSED

6. Conclusion and Analysis

In this lab, property-based fuzz testing with Hypothesis successfully identified critical input validation vulnerabilities that standard manual testing missed. Because developers usually test with expected data formats, edge cases such as None inputs, empty delimiter-separated lists, and incomplete record formats often slip into production. Fuzz testing proved to be an effective automated tool for uncovering these hidden crash triggers. By combining Hypothesis with defensive programming practices—including rigorous type checks, bounds checks, and resilient type casting—the application was successfully hardened to guarantee that it remains stable and crash-free under all operating conditions.